



Licenciatura Engenharia Informática e Multimédia
Instituto Superior de Engenharia de Lisboa
Ano letivo 2024/2025

Modelação e Simulação de Sistemas Naturais
Relatório: Projeto Final

Turma: 31D

Nome: João Ramos

Número: 50730

Nome: Miguel Alcobia

Número: 50756

Professor: Arnaldo Abrantes

Data: 19 de Janeiro de 2025

Índice

Lista de Figuras	3
Introdução	4
Jogo e Mecânicas	5
Contexto da Ideia e História Jogo	5
Implementação	6
Geração do Terreno	6
Comportamento dos Monstros	6
Contador de Kills	7
Comportamento e Mecânicas do Ninja	8
Diagrama UML	10
Descrição da Classe Mover	11
Descrição da Classe Body	11
Descrição da Classe ParticleSystem	11
Descrição da Classe DNA	11
Descrição da Classe Eye	11
Descrição da Classe Behavior	11
Descrição da Classe Seek	12
Descrição da Classe Arrive	12
Descrição da Classe AvoidObstacles	12
Descrição da Classe CellularAutomata e MajorityCA	12
Descrição da Classe Cell e MajorityCell	12
Descrição da Classe Terrain	12
Descrição da Classe Entity	12
Descrição da Classe Player	13
Descrição da Classe WorldConstants	13
Descrição da Classe GameApp	13
Conclusões	15
Bibliografia e Ferramentas	16

Lista de Figuras

Figura 1 - Captura de ecrã do Vampire Survivors	5
Figura 2 - Sprites dos Monstros	7
Figura 3 - Captura de Ecrã com o contador destacado	8
Figura 4 - Sprites das skins do Ninja	9
Figura 5 - Captura de ecrã com a área das Skins destacada	9
Figura 6 - UML	10
Figura 7 - Ecrã do Menu do Jogo	13
Figura 8 - Ecrã dos Créditos do jogo	14

Introdução

Neste projeto final, os alunos tinham como objetivo desenvolver um jogo/simulação, aplicando os conhecimentos adquiridos ao longo do semestre, tais como:

- “1. Modelação baseada em Stocks & Flows;*
- 2. Modelação baseada em agentes (e.g., Boids);*
- 3. Geração procedimental de conteúdos com objetos fractais (e.g., árvores) e autómatos celulares (e.g., terrenos);*
- 4. Simulação do movimento de objetos usando as leis da física;*
- 5. Modelação de comportamentos individuais e de grupo em agentes autónomos;*
- 6. Simulação da Seleção Natural que permite explicar a evolução das características genéticas de uma população (brevemente);”*

No desenvolvimento do projeto, os alunos devem usar a experiência das aulas de laboratório e dos Trabalhos Práticos, ou seja, utilizar as bibliotecas do Processing para desenvolver aplicações gráficas e interativas e ferramentas de simulação como o Silico.

Os alunos decidiram, com o apoio do professor, fazer um jogo inspirado no videojogo Vampire Survivors.

Jogo e Mecânicas

Contexto da Ideia e História Jogo

Como apresentando no final da Introdução, o grupo decidiu fazer um jogo baseado num videojogo chamado Vampire Survivors.

Este jogo foi desenvolvido por uma única pessoa, Luca Galante, também conhecido como “poncle” e foi lançado em 2022. Trata-se de um jogo do género *Roguelike* e de tiro do tipo *shoot'em up*. Neste jogo somos um caçador de monstros que mata hordas de monstros e ao matá-los vamos adquirindo experiência para subir de nível.

Para o projeto decidimos adotar mais ou menos a mesma sinopse. Em vez de um caçador somos um ninja que tem como objetivo sobreviver a waves ao matar monstros e à medida que evoluímos é nos permitido vestir outras roupas como recompensa (roupa de ferro, ouro e diamante). O jogo acaba quando o ninja consegue derrotar os últimos dragões que vêm ao seu encontro nas masmorras.

Para tornar este jogo realidade, o grupo decidiu recorrer às seguintes partes da matéria:

- **Autómatos Celular** (Criação do Mundo),
- **Agentes Autónomos** (Player e Monstros),
- **Comportamentos Individuais e de Grupo** (Player e Monstros),
- **Sistemas de Partículas** (Tiros do Player),
- **Simulação de Objetos com Leis da Física** (deslocação do Player e dos Monstros)



Figura 1 - Captura de ecrã do Vampire Survivors

Implementação

Geração do Terreno

O terreno é baseado num autômato celular 2D e é gerido na classe `Terrain` que deriva da classe `MajorityCA`, isto porque a regra da maioria é aplicada ao autômato para a criação da masmorra (o mundo do jogo). Desta forma, conseguimos criar um mundo coeso e que faça sentido, sem ter por exemplo várias células que representam a água espalhadas isoladamente pelo mapa enquanto as podemos ter todas juntas.

Cada célula do autômato é um objeto da classe `Patch` que deriva da classe `MajorityCells`. Existem 4 tipos de Patches, cada um corresponde a um tipo de terreno: `Empty`, `Obstacle`, `Mud` e `Water`. Patches dos tipos `Obstacle`, `Mud` e `Water` diminuem a velocidade do jogador pela metade enquanto ele se encontrar em cima das mesmas.

Comportamento dos Monstros

Neste jogo temos os seguintes monstros: Morcegos, Esqueletos, Zombies, Monstros das Profundezas e Dragões.

Os monstros aparecem em hordas (*waves*) e perseguem o jogador. Fez-se a seguinte divisão entre monstros e *waves*:

- **Até o nível 5**, surgem 5 morcegos,
- **Do nível 6 ao 9**, aparecem 3 zombies e a cada leva adiciona-se um zombie ao grupo,
- **Do nível 10 ao 14**, aparecem 3 esqueletos e a cada leva adiciona-se um esqueleto ao grupo,
- **Do nível 15 ao 19**, surgem 3 Monstros das Profundezas e a cada leva adiciona-se um monstro ao grupo,
- **Do nível 20 ao 25** surgem 3 Dragões e a cada leva adiciona-se um dragão ao grupo.

Assim que se mata o último dragão o jogo acaba.

Todos os monstros fazem partes de **Flocks**, que já foram utilizados no trabalho prático 2, mas com algumas adições no código existente. No construtor da classe `Flock` acrescentaram-se dois atributos `String`: `img_src` e `type`.

O primeiro atributo serve para passar o *path* da imagem que os boids do `Flock` devem assumir e o segundo serve para indicar o tipo de monstros do `Flock` (`bat`, `zombie`, `sklt`, `seamonster` e `dragon`) para incrementar-se o contador de kills do monstro correto.

Na classe `GameApp`, que gere a lógica do jogo, estão definidas para cada flock a quantidade de boids iniciais, o dano que dão ao jogador e os pontos de experiência que oferecem ao jogador. Na própria classe `Flock` também se adicionou uma nova função `Hunt()`, que adiciona o comportamento `Seek` a todos os boids do `Flock`.

O seek que eles executam têm como target o Player.



Figura 2 - Sprites dos Monstros

Contador de Kills

Do lado direito da simulação desenhou-se um contador para ver quantos monstros o jogador matou de cada tipo. Para tal, desenvolveu-se a função `drawKillCount()` que coloca as imagem dos sprites dos monstros alinhados com o respetivo contador cujo valor vem da variável `kill<NOME DO MONSTRO>`.

A morte de um monstro acontece no método `checkCollisions()` que fica na `GameApp`, a correr na função `drawGame()`, que por sua vez corre na função `draw()`.

Neste método, cria-se uma lista para remover os boids que morreram. Para saber se um monstro morreu, percorre-se todos os boids do flock e todas as partículas que o Player dispara. Calculamos a distância entre o monstro e a partícula e caso seja inferior ao raio do Boid é detetada uma colisão entre o monstro e a partícula e considerasse o monstro como morto.

Após a deteção da colisão, adiciona-se o boid à lista de boids a remover e verificamos o tipo de monstros do boid para incrementar o respetivo contador.

A seguir demonstra-se como esta lógica foi implementada no código, exemplificando a colisão para o caso dos morcegos.

```
List<Boid> boidsToRemove = new ArrayList<>();
for (Boid boid : flock.getBoidList()) {
    for (Particle particle : particleSystem.getParticles()) {
        float distance = PVector.dist(boid.getPos(), particle.getPos());
        if (distance < boid.getRadius()) {
            boidsToRemove.add(boid);
            if (flock.getType() == "bat"){
                killBAT++;
            }
        }
    }
}
```

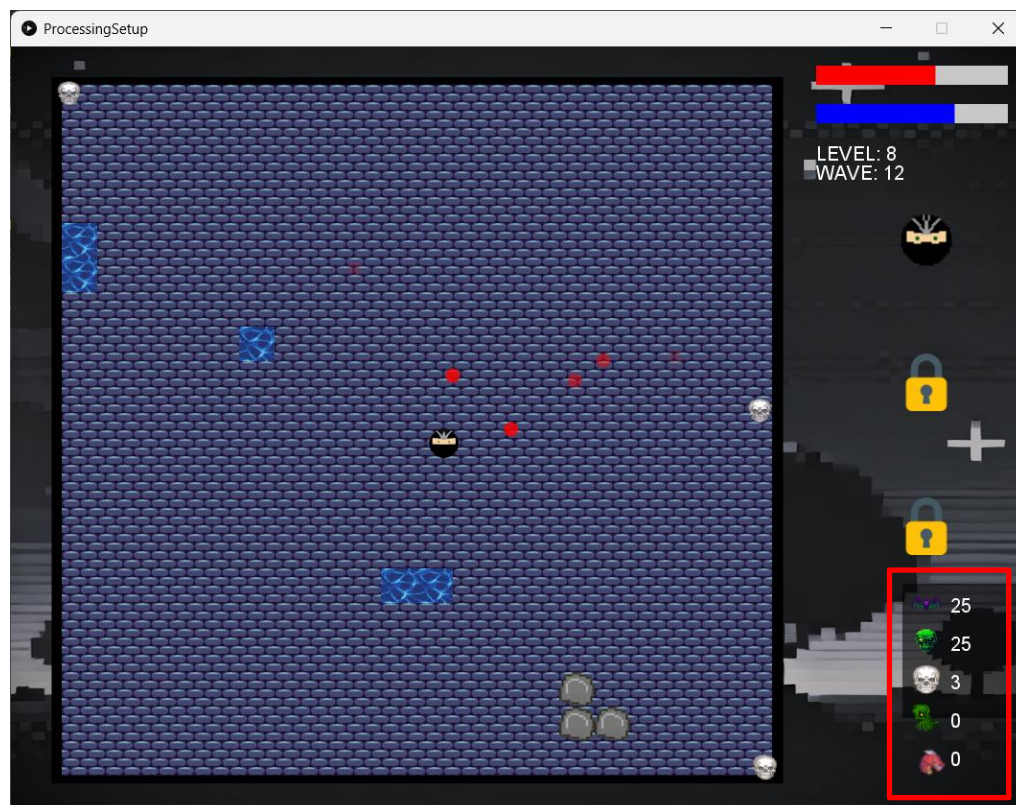


Figura 3 - Captura de Ecrã com o contador destacado

Comportamento e Mecânicas do Ninja

O Ninja começa com 20 pontos de vida (HP) e zero de experiência (XP). Para movimentá-lo, o jogador deve clicar no destino pretendido para que o Player faça Arrive a essa posição. Optou-se pelo comportamento Arrive para evitar o efeito “pêndulo” que ocorria com o Seek em situações em que o ninja chegava à posição do target com grande velocidade.

Para desbloquear roupas (skins) e aumentar os seus stats, o jogador deve aumentar de nível e para isso mata os inimigos.

Cada inimigo oferece um número específico de pontos de XP consoante o seu tipo. A quantidade de xp necessária para o Player subir de nível é calculada na função `calcMaxXp()`, que ao receber o nível do jogador multiplica-o por 10 e soma ao produto mais 10 para obter o valor desejado. O código abaixo mostra a implementação deste conceito:

```
public int calcMaxXp(int lvl){
    this.setMaxXp(10+(lvl*10));
    return 10+(lvl*10);
}
```

A função `updateLevel()` da classe Player recebe o sistema de partículas, que são os tiros do ninja, e é responsável por implementar a lógica de desbloquear as roupas ao personagem e atualizar os valores de vida e

experiência. A divisão feita para as recompensas ao subir de nível foi a seguinte:

- **Em todos os níveis** recalcula-se o valor máximo de pontos de XP ao obter,
- **Em todos os níveis** o jogador recebe 10 pontos de vida e a sua vida máxima também aumenta 10 pontos,
- **No nível 5**, o ninja desbloqueia a roupa de Ferro e pode disparar 2 balas por segundo,
- **No nível 10**, o ninja desbloqueia a roupa de Ouro e pode disparar 3 balas por segundo,
- **No nível 15**, o ninja desbloqueia a roupa de Diamante e pode disparar 4 balas por segundo.

Do lado direito do jogo, sobre o contador de kills, desenharam-se as roupas do personagem com a função `drawSkins()` para permitir ao jogador ter uma percepção do seu progresso no jogo e conseguir trocar de roupa à medida que as vai desbloqueando.

Quando a roupa está bloqueada é desenhada um cadeado e quando se clica na zona do cadeado é executada a função `checkClickSkin`, que recebe as coordenadas de onde está a imagem da skin ou do cadeado, a String com o *path* da skin correspondente e verifica se a skin está desbloqueada para o jogador com a função `isSkinAvaliable()`, caso esteja, troca-se a shape do Boid do jogador para ficar com a roupa desejada. A função `isSkinAvaliable()` retorna valores booleanos e recebe o *path* da *skin* pretendida e, consoante esse valor, verifica o nível do personagem para ver se este já tem acesso a essa roupa.

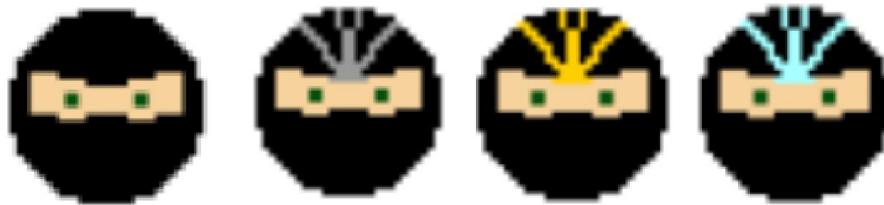


Figura 4 - Sprites das skins do Ninja

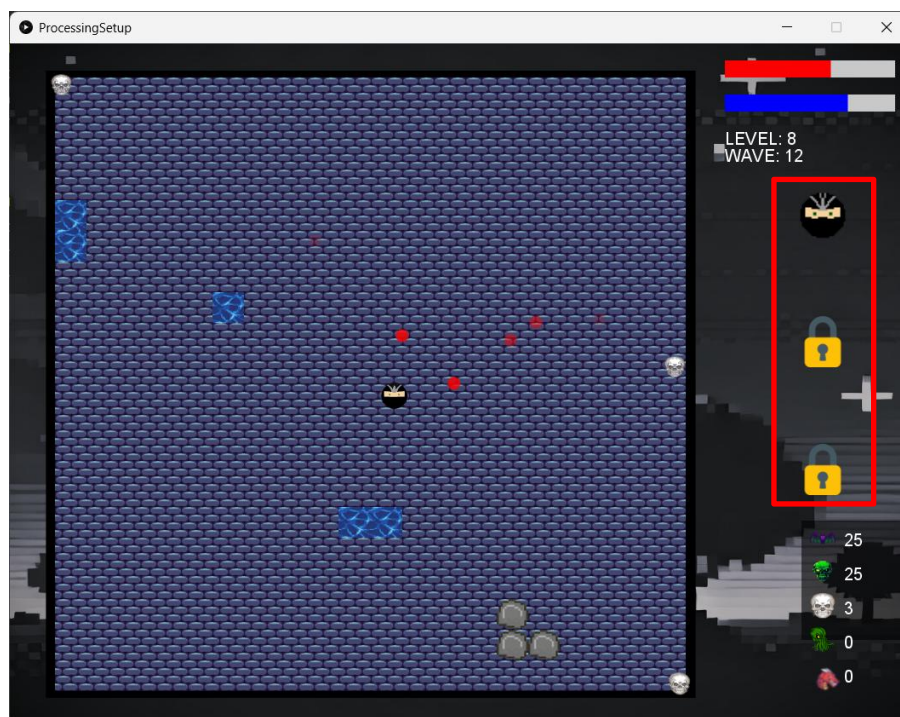


Figura 5 - Captura de ecrã com a área das Skins destacada

Desenvolveu-se a função `drawLifeBar()` e `drawExpBar()` para desenhar as barras de vida e experiência e preenchê-las tendo por base os respetivos valores. A barra de vida é preenchida com a cor vermelha, enquanto a barra de experiência é colorida a azul.

O jogador morre quando a sua vida chega a zero, Este controlo é feito na classe `GameApp` na função `checkPlayerCollisions()`, onde se define um intervalo para o jogador receber dano para evitar que a vida dele seja constantemente diminuída pelos monstros. Caso contrário o dano era sofrido a cada *frame*. A lógica desta função é semelhante à que controla a morte dos monstros, mas desta vez percorremos todos os boids e se a distância for menor que a definida e o tempo entre o dano for respeitado, aplica-se o dano ao jogador e atualizado o tempo da última vez que o jogador levou dano.

Diagrama UML

No final do projeto, produziu-se o diagrama de classes do trabalho e o resultado foi o seguinte:

(Devido à dimensão do UML, a resolução ficou má para a leitura do mesmo. Por esse motivo, o UML está presente na pasta “mod” no ficheiro zip do trabalho)

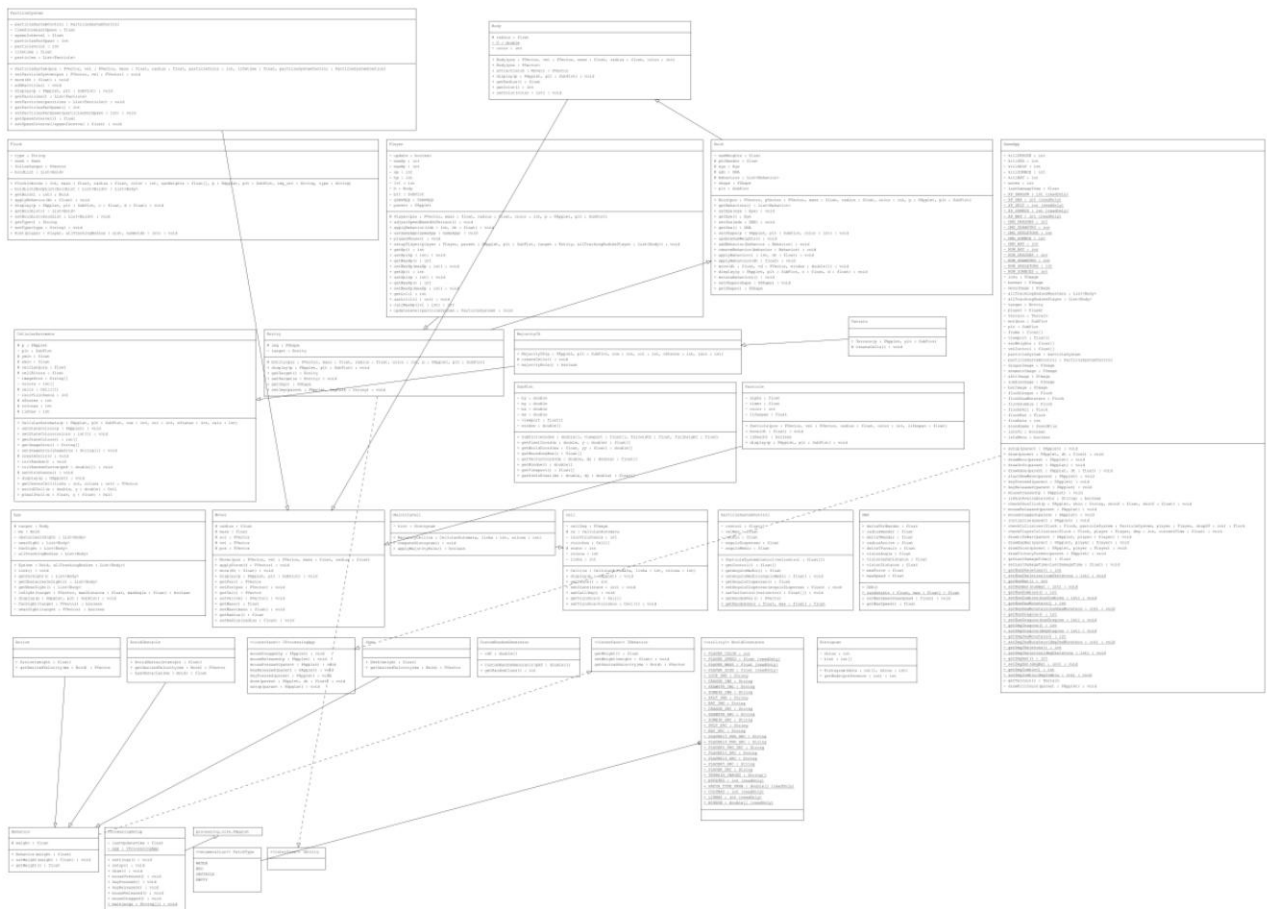


Figura 6 - UML

Descrição da Classe Mover

Esta classe pretende simular um corpo/objeto que se movimentará e tem como atributos: o vetor de posição (pos), um de velocidade (vel), um de aceleração (acc), um valor float que diz respeito à massa (mass) e um valor float que diz respeito ao seu raio (radius).

Descrição da Classe Body

A classe Body é a classe que dará origem à classe Boid, que por sua vez originará a class Entity. A classe Body herda os atributos e os métodos da classe Mover. Tem como atributos próprios a cor (color) e por fim um float que corresponde ao seu raio.

Descrição da Classe ParticleSystem

Esta classe pretende simular um sistema de partículas e tem como atributos próprios: a lista de partículas do sistema (particles), um float com o tempo de vida da partícula (lifetime), um int para a sua cor (particleColor), um sistema de controlo do ParticleSystemControl (particleSystemControl) e um valor float que diz respeito ao raio da partícula (radius).

Descrição da Classe DNA

A classe DNA representa o DNA de uma criatura com as suas características próprias, apresentadas nos atributos da classe. Entre elas estão a Velocidade Máxima, a Força Máxima, Distância da Visão, Distância Segura da Visão, Ângulo da Visão e os parâmetros para o funcionamento dos comportamentos.

Descrição da Classe Eye

Esta classe pretende simular um olho/campo de visão de um Boid e tem como atributos: 4 Listas de Body: allTrackingBodies (todos os Bodies que o olho vê); farSight (os bodies que estão dentro do campo visão, mas ainda longe); nearSight (os que estão muito próximo do campo de visão) e obstaclesInSight (os obstáculos que o Boid vê). Além disso, tem o Boid me (o próprio Boid) e o target (Boid alvo).

Descrição da Classe Behavior

A classe Behavior dará origem aos comportamentos, tendo como único atributo um valor float que apresenta o peso desejado que o comportamento deverá ter (weight).

Descrição da Classe Seek

A classe Seek implementa o comportamento Seek, que consiste na perseguição de um alvo, até chegar à sua posição.

Descrição da Classe Arrive

A classe Arrive implementa o comportamento Arrive, que consiste na desaceleração à medida que o corpo se aproxima do seu alvo.

Descrição da Classe AvoidObstacles

A classe AvoidObstacles implementa o comportamento AvoidObstacles, que consiste no desvio de obstáculos presentes no terreno, invertendo o sentido do deslocamento.

Descrição da Classe CellularAutomata e MajorityCA

A classe CellularAutomata implementa um autômato celular, enquanto a classe MajorityCA herda da CellularAutomata, mas implementa a regra da maioria.

Descrição da Classe Cell e MajorityCell

A classe Cell implementa uma célula de um autômato celular, enquanto a classe MajorityCell herda da Cell, mas possui um histograma para ser possível implementar a regra da maioria.

Descrição da Classe Terrain

A classe Terrain é uma MajorityCA, com o intuito de gerar um terreno, com diversos tipos, utilizando a regra da maioria.

Descrição da Classe Entity

A classe Entity é um Boid, que será utilizada como base para a classe Player.

Descrição da Classe Player

A classe Player é uma Entity, com atributos relacionados a mecânicas do jogo (vida e xp). Contem métodos que aplicam mudanças quando o player atinge um novo nível e também métodos que calculam os novos patamares de experiência que o player precisa atingir. Por fim, tem um método que ajusta a sua velocidade tendo em conta o tipo de terreno em que o player se encontra.

Descrição da Classe WorldConstants

A classe WorldConstants é uma classe que guarda em si valores constantes para a construção e funcionamento do sistema.

Descrição da Classe GameApp

A classe GameApp é a classe onde se gere a maior parte das mecânicas do jogo. Esta classe tem como principais objetivos dar display ao jogo, criação de novas waves, deteção de colisões e detetar em que estado a simulação se encontra (Menu, Jogo e Painel de Créditos). Algumas das variáveis e funções que estão nesta classe poderiam estar noutras classes para uma melhor organização e modelação do trabalho. Contudo, com o tempo disponível, o grupo deu prioridade a colocar o máximo das mecânicas que pretendia e acabou por deixar este aspeto de lado.

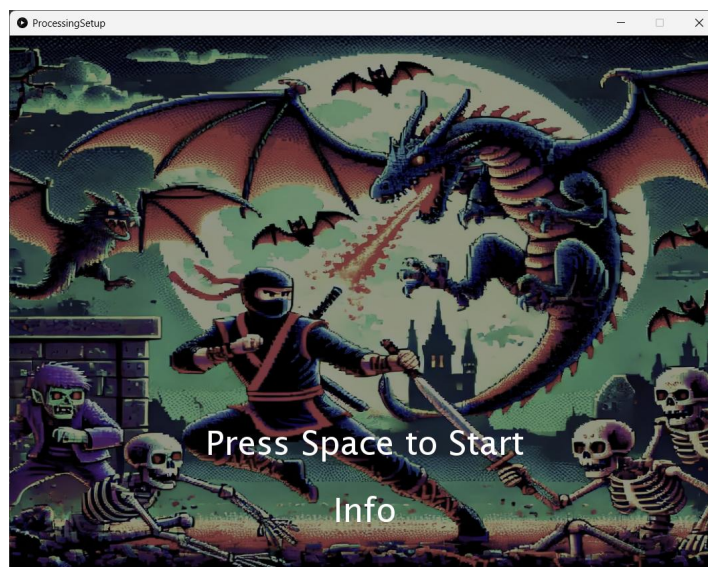


Figura 7 - Ecrã do Menu do Jogo

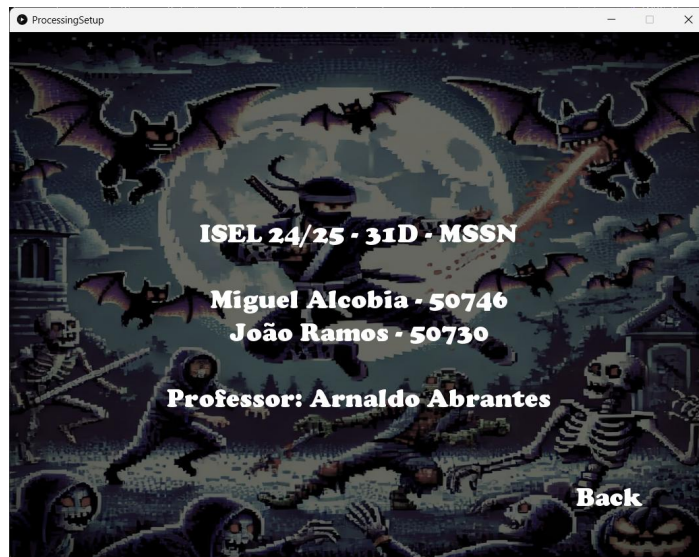


Figura 8 - Ecrã dos Créditos do jogo

Conclusões

Com a realização deste projeto, percebemos da diferença que é fazer um jogo do zero sem qualquer ajuda que um motor gráfico como a Unity oferece nestes casos. Embora seja muito mais trabalhoso, desta forma temos um maior controlo sob o nosso projeto e conseguimos perceber mais rapidamente a origem dos problemas.

Como foi apontado no tópico anterior o grupo desfrutou de pouco tempo para realizar o trabalho e isso refletiu-se nalgumas situações como a organização do código. Muitas das variáveis e funções que são declaradas na classe GameApp deveriam estar presentes em outras classes, contudo deu-se prioridade a implementar o máximo de mecânicas para entregar uma versão mais próxima daquela que era a ideia inicial do grupo.

Embora tenhamos atingindo a maioria dos nossos objetivos neste trabalho, algumas mecânicas acabaram por ficar de lado: O grupo gostaria de ter feito uma mecânica, na qual apenas os morcegos e os dragões voavam pelo mapa sem serem afetados pelo terreno. Cada monstro teria uma habilidade especial o que lhe traria algo único. Por exemplo, os zombies teriam a habilidade VIRUS que diminuiria a velocidade do player durante um pequeno período de tempo como também o afetaria com dano por segundo durante um breve período.

Outra ideia deixada de lado foi o facto de que, em estágios de iniciais, o viewport seria apenas uma porção do mapa e quando o jogador tocasse numa das margens do subplot, o mapa expandiria nessa direção.

Mesmo com estas ideias fora do projeto final, o grupo concluiu que conseguiu alcançar um resultado satisfatório e interessante ao ter conseguido realizar um projeto com uma proposta mais única do padrão pedido para o projeto (Ecossistema de animais).

Bibliografia e Ferramentas

Consulta:

Abrantes, Arnaldo; Júnior, Carlos e Vieira, Paulo. (2023). Física e Movimento - Moodle

Vídeos gravados pelo Prof. Arnaldo Abrantes - Moodle

Pixel Art (Para pequenos retoques a assets encontrados):

Descottes, J. (2019). *Piskel - Free online sprite editor*. Piskelapp.com. <https://www.piskelapp.com/>

Inspiração:

Vampire Survivors on Steam. (n.d.). Store.steampowered.com.

https://store.steampowered.com/app/1794680/Vampire_Survivors/